

Chapter 3. Spring boot and Data



이영곤



- ✓ What is Spring Boot?
 - Spring Boot is an open-source Java framework that simplifies the development of web applications, REST APIs, and enterprise systems by reducing configuration and setup work required by the traditional Spring Framework.
 - It allows developers to create stand-alone, production-ready applications quickly with minimal configuration.

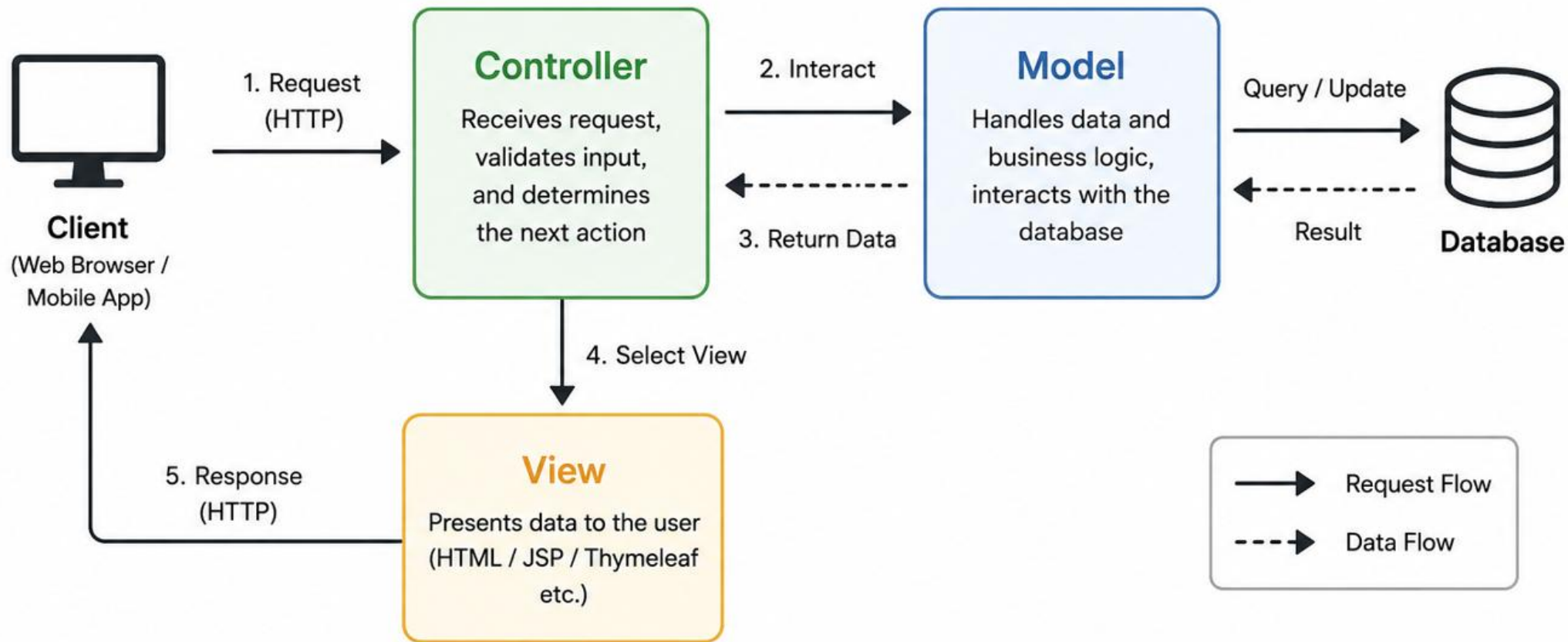
Key Features of Spring Boot

- ✓ Auto Configuration
 - Automatically configures Spring components based on project dependencies.Reduces XML and Java configuration.
- ✓ Embedded Web Server
 - Includes built-in servers such as: Tomcat (default), Jetty, Undertow
 - No need to install a separate application server.

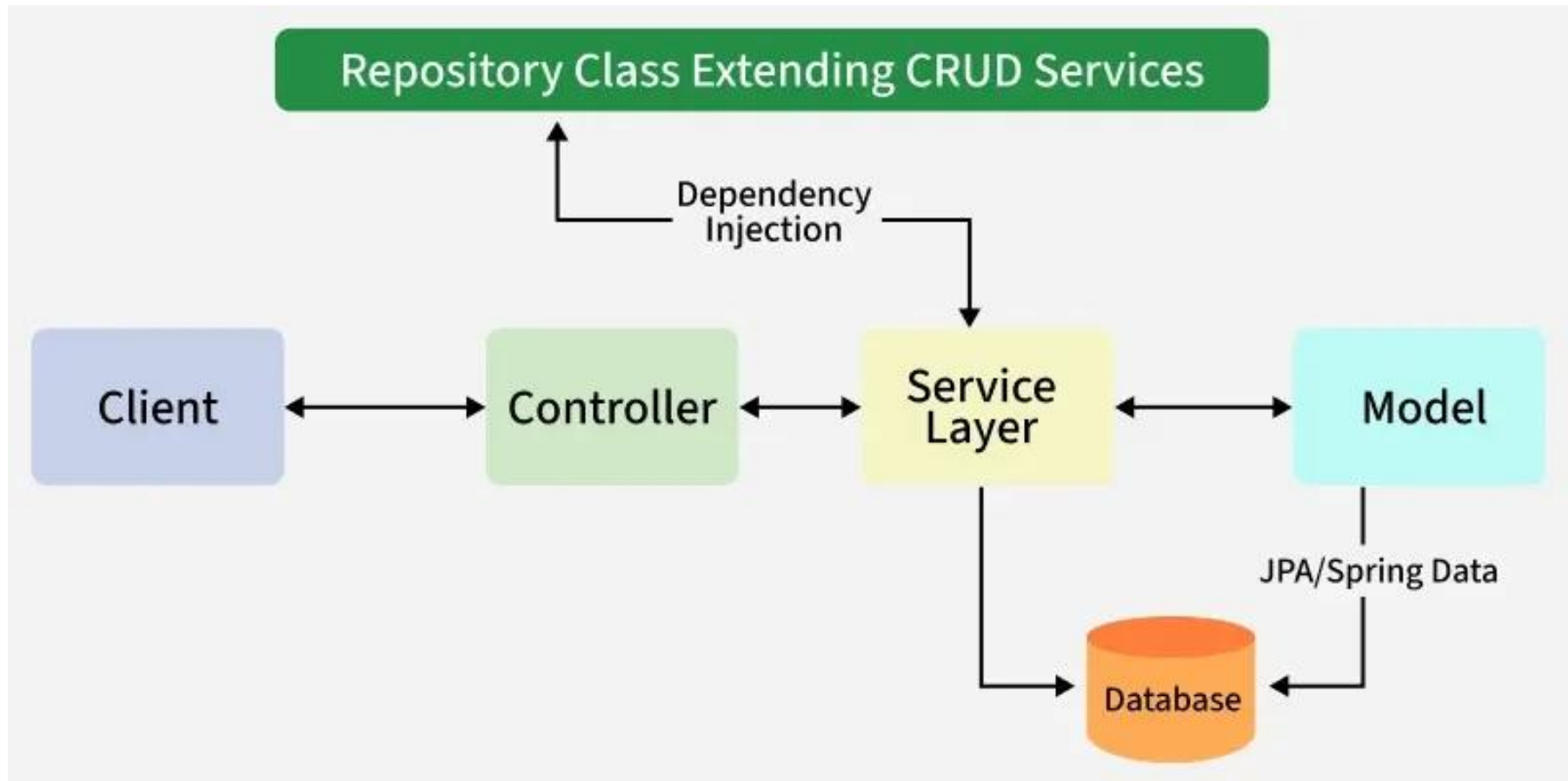
Key Features of Spring Boot

- ✓ Simplifies dependency management.
- ✓ Convention over Configuration
 - Provides sensible default settings.
 - Developers only configure what is necessary.
- ✓ Production Ready Features

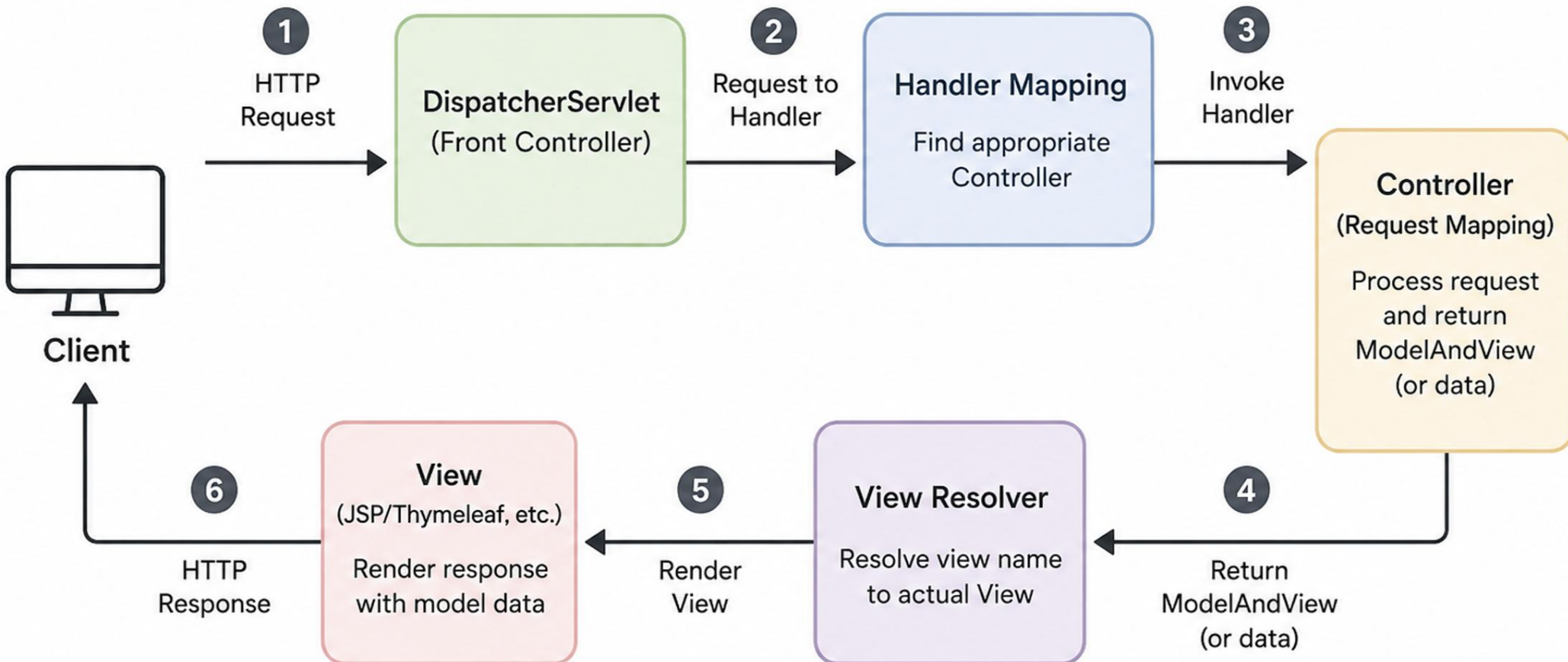
Model-View-Controller(MVC) Pattern



General architecture of Spring Boot app



» Spring boot MVC Request flow



» Starter

- Provides dependencies that the application must include
- Provides auto-configuration
- spring-boot-starter-*
- Spring Boot is deployed in an independent web container in the form of a Jar file
- Microservices should not share DB or web containers

» main class

- Add @SpringBootApplication annotation has the same meaning as @Configuration, @EnableAutoConfiguration, and @ComponentScan
- Includes spring-boot-starter-web

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

» App build

- Must provide configuration responsible for including all dependencies into an executable Jar (fat Jar) during app build.
- This is performed by the spring-boot-maven-plugin defined in pom.xml
- Includes spring-core, spring-app, spring-context, Spring Boot, embedded Tomcat, logback, Log4j, Slf4j, Jackson lib, etc.

```
<plugin>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-maven-plugin</artifactId>  
</plugin>
```

» Customizing the Configuration File

- Provides configuration information through application.yml

```
server:
  port: ${port:2222}

spring:
  application:
    name: first-service

logging:
  pattern:
    console: "%d{HH:mm:ss.SSS} %-5level %logger{36} - %msg%n"
    file: "%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"
# level:
#   org.springframework.web: DEBUG
  file: app.log

management:
  security:
    enabled: false
```

» Restful web service

- The Jackson library, which serializes and deserializes JSON messages, is included in the spring-boot-starter-web.
- All you need to do is declare a model class
- `@RestController` sets incoming HTTP requests to the empty class of controllers
- `@RequestMapping`: Used to correspond to controller methods and HTTP
- `@GetMapping`, `@PostMapping`, `@DeleteMapping` = `@RequestMapping(method=RequestMethod.GET) ..`
- `@RequestParam`: Bind the path and input of the request to Object
- `@RequestBody`: Bind Input JSON to Object using Jackson Library

» Spring boot Actuator

- Provides endpoint for monitoring and interacting with apps
- /beans: Displays a list of all the spring bins initialized in the app
- /env: Displays a list of configurable environmental properties for the spring
- /health: displaying the status of the app
- /info: Displays random information in the app
- /loggers: displaying logger configuration information for the app
- /metrics: Displays the app's metric information (memory, thread, rest response time, etc.)
- /trace: displaying trace information
- To disable security in a project, manage it in the application.yaml

← → ↺ 🏠 ⓘ localhost:2222/metrics

```
{ "mem": 626192, "mem.free": 512993, "processors": 8, "instance.uptime": 1514270, "uptime": 1529231, "systemload.average": -1.0, "heap.committed": 569344, "heap.inuse": 56850, "nonheap": 0, "threads.peak": 35, "threads.daemon": 29, "threads.totalStarted": 39, "threads": 31, "classes": 8451, "classes.loaded": 8451, "gc.ps_marksweep.time": 130, "http.sessions.max": -1, "http.sessions.active": 0, "gauge.response.beans": 30.0, "gauge.response.swagger-resources.configuration.resources.configuration.ui": 80.0, "gauge.response.webjars.star-star": 10.0, "gauge.response.v2.api-docs": 133.0, "gauge.response.swagger-resources": 13.0, "star": 3.0, "gauge.response.star-star.favicon.ico": 24.0, "counter.status.200.person.id": 2, "counter.status.200.swagger-resources.configuration.security.resources": 1, "counter.status.200.person": 1, "counter.status.500.person.id": 2, "counter.services.person.add": 1, "counter.status.200.beans": 1, "counter.star": 1, "counter.status.200.swagger-resources.configuration.ui": 1, "counter.status.200.v2.api-docs": 1, "counter.status.200.webjars.star-star": 23 }
```

» Developer Tools

- Automatically restart when a file on the class path changes
- Include spring-boot-devtools dependency
- Include in Exclusion to avoid restarting on change

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-devtools</artifactId>  
    <optional>true</optional>  
</dependency>
```

```
spring:  
  application:  
    name: first-service  
  devtools:  
    restart:  
      exclude: static/**
```

» Developer Tools

- SpringCloud gets its configuration from a remote server. To this end, you need to set the minimum settings required in the app in the bootstrap context, i.e., the app name and configuration address, so that the configuration can be read from an external source.
- Bootstrap context uses bootstrap.yml
- Set spring.cloud.bootstrap.enabled to false to disable bootstrap
- Bootstrap configuration file name can be changed from spring.cloud.bootstrap.name .Change location to location

```
spring:
  application:
    name: first-service
  cloud:
    config:
      uri: http://192.168.99.100:8888
```

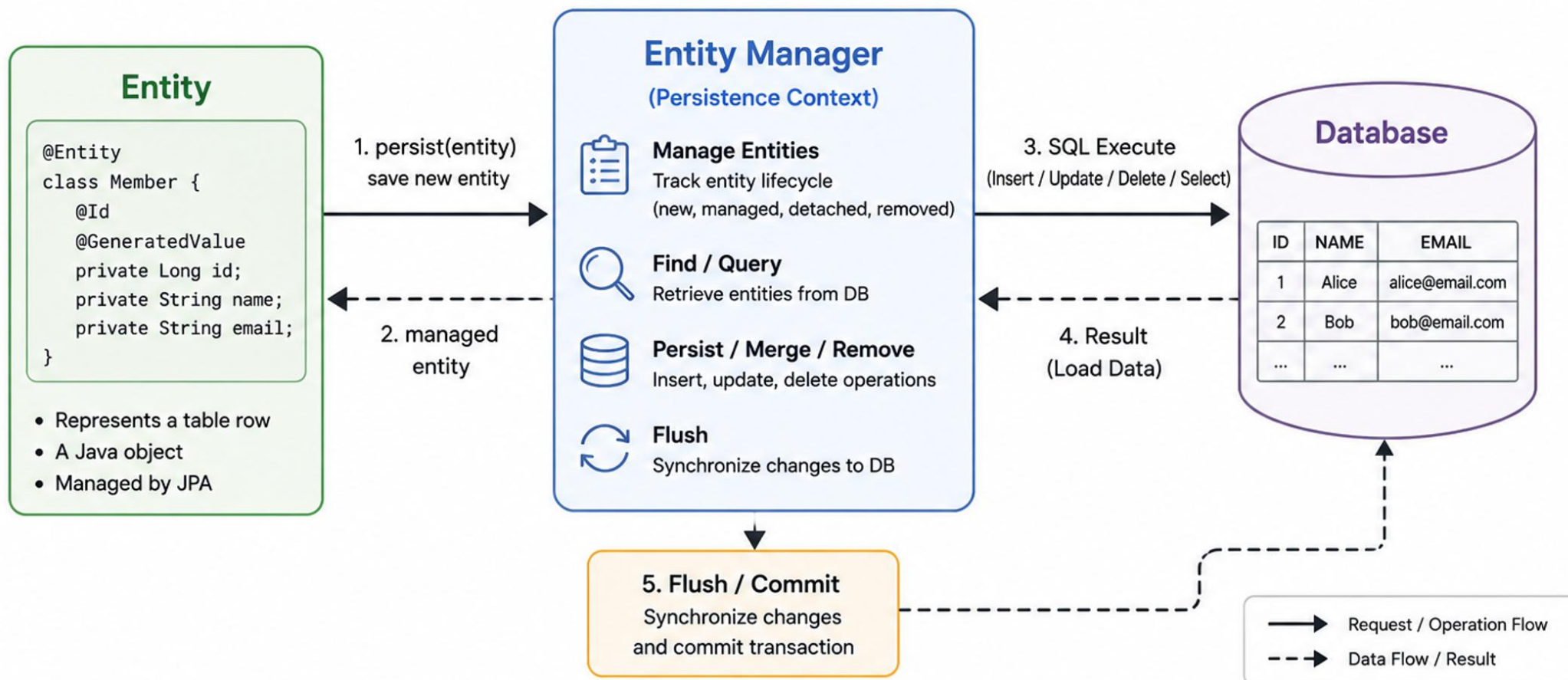
» Entity

- An Entity is a lightweight Java object that represents a table in a relational database.
- Each entity object corresponds to a row (record) in a database table, and its fields correspond to the table's columns.

» Entity Manager

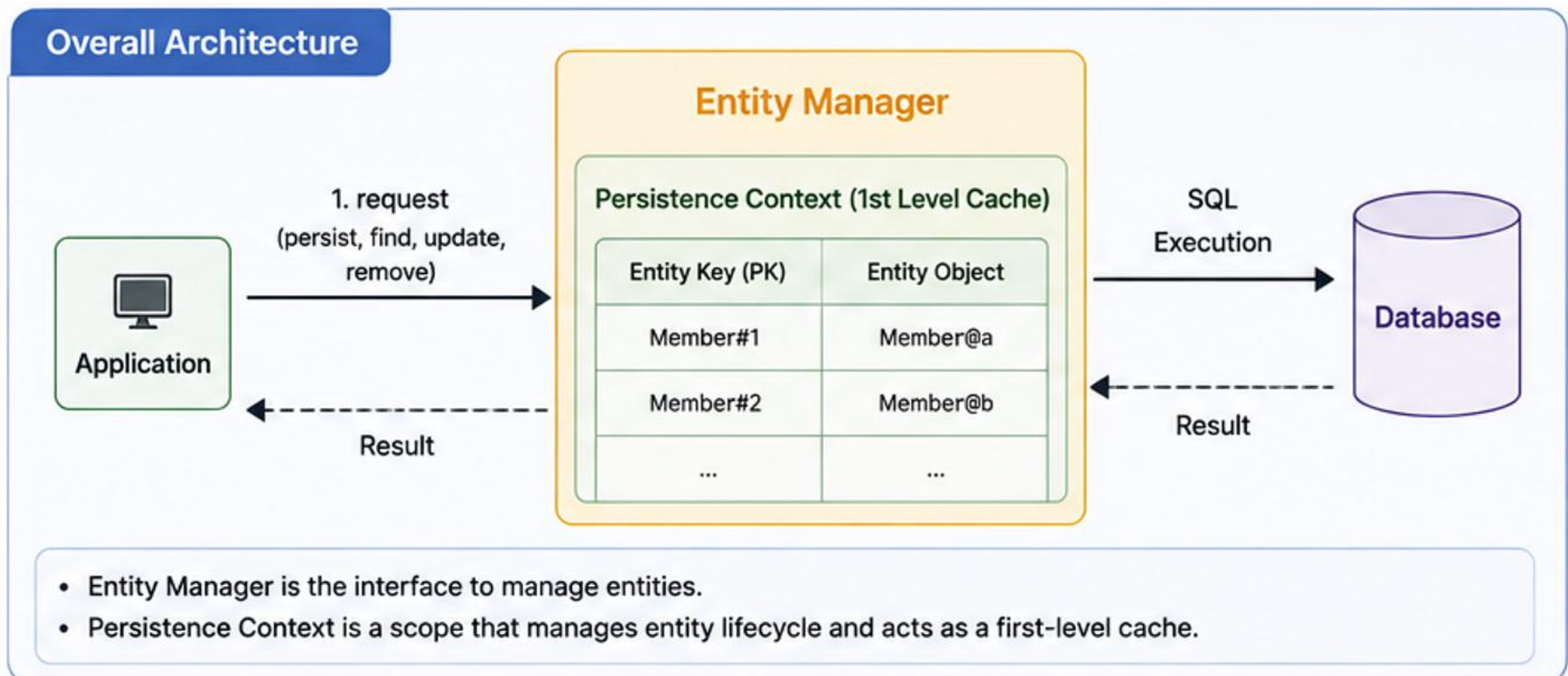
- Manage entities to create, modify, and delete objects between DBs and apps
- The entity manager factory creates the entity manager
- Create and manage only one entity manager factory inside the spring boot and use entity manager using `@PerformanceContext` or `@Autowired`

Relationship of entity & entity manager



» Persistence Context

- The entity manager stores the entity in a persistent context
- A persistent context is a virtual space that manages entities
- Features primary cache, write delay, change detection, delay loading



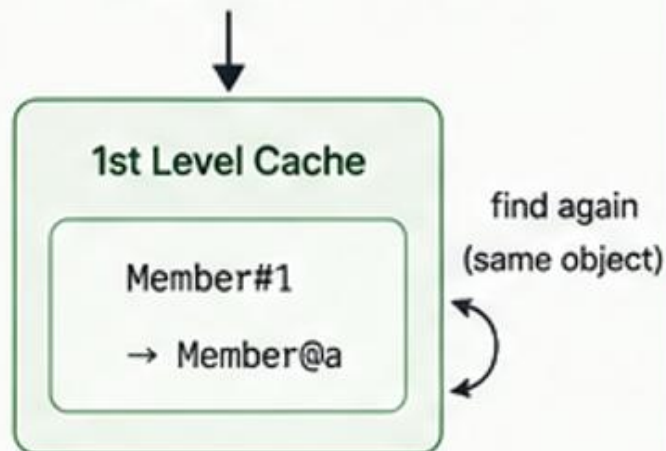
» 1st-Level Cache

- The key in the cache is an identifier with the entity's @Id and the value is the entity
- When looking up an entity, look up in the primary cache and return it if there is a value, read it from the DB, save it in the cache, and return it

Example

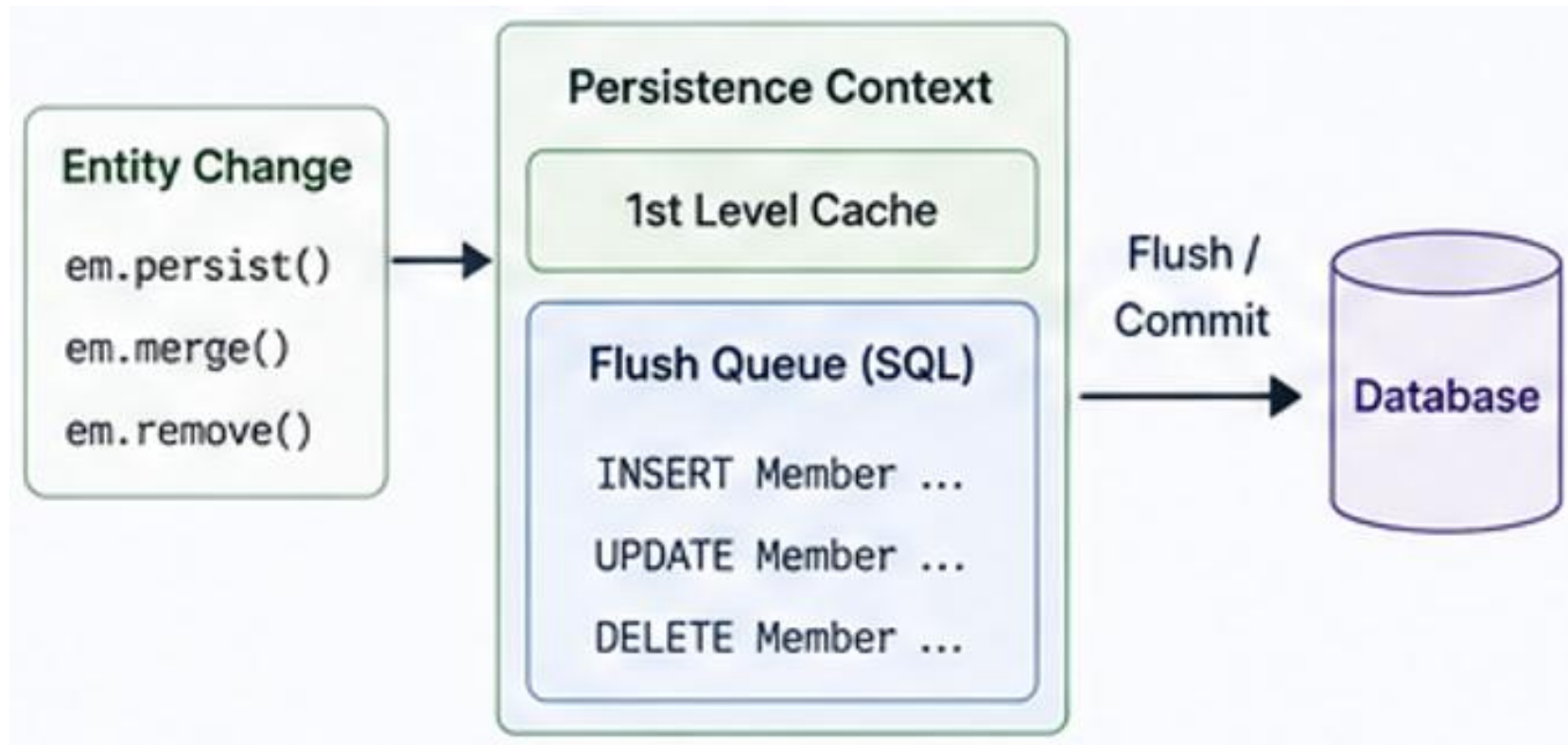
```
Member m1 = em.find(Member.class, 1L);  
Member m2 = em.find(Member.class, 1L);  
  
m1 == m2    // true (same instance)
```

find(Member, 1L)



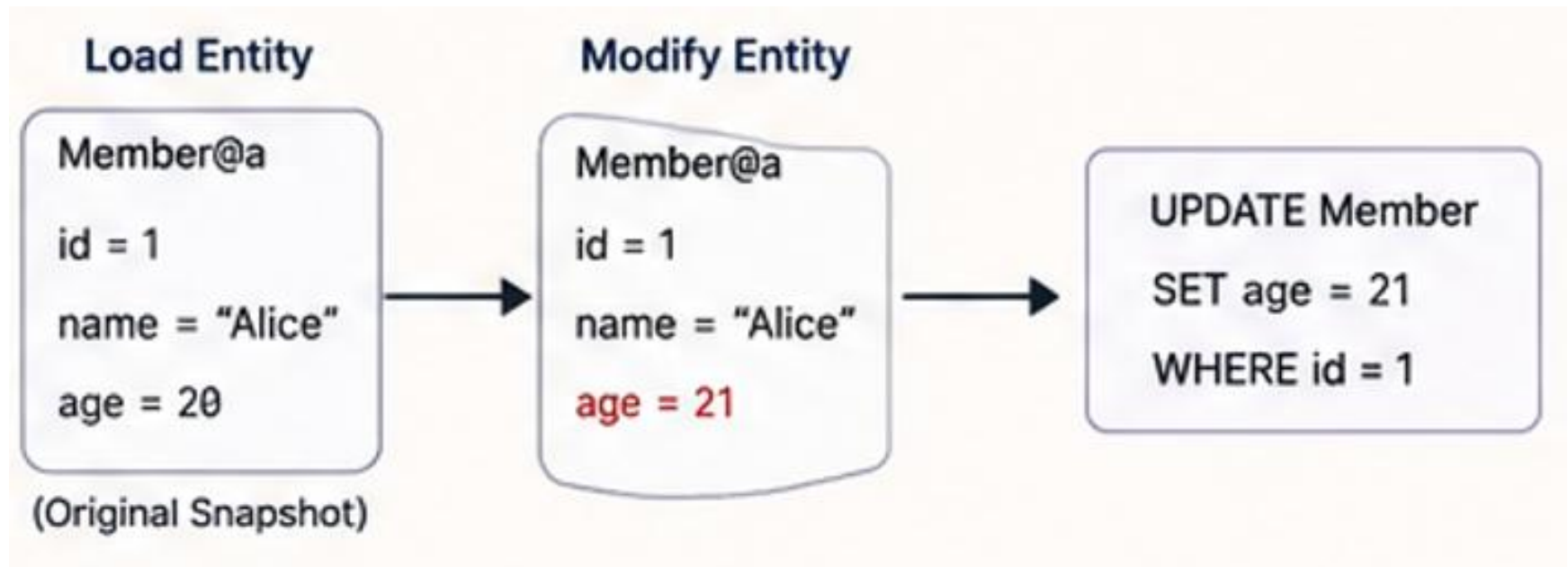
» Write-Behind

- Collect queries without sending queries to the DB until the transaction is committed, and then execute them at the same time



» Dirty Checking

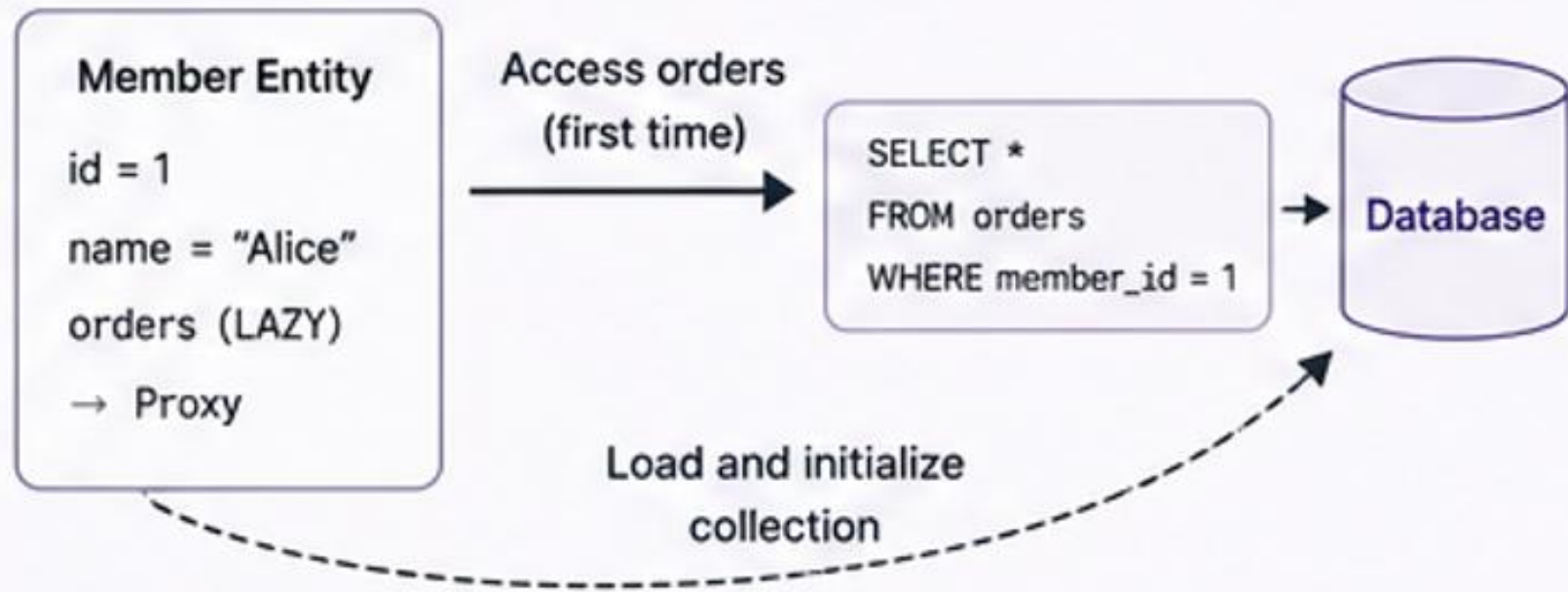
- When committing a transaction, compare the value of the entity stored in the primary cache with the value of the current entity. If there is a change, the change is detected and automatically reflected in the DB



» Lazy Loading

- This means querying the data requested by the query by sending the query when needed, rather than loading it directly into the app

Example



» Spring Data

- Abstracts DB usage at class level to focus on business logic
- Use spring data through the interface provided by the spring data

» Spring Data JPA

- Adding useful skills from JPA in common features of spring data
- Create an interface that acts as a repository and simply look up, modify, create, and delete DB tables

@Repository

```
public interface BookRepository extends CrudRepository<Book, Integer> {  
}
```


» Spring Data JPA

```
@Service
public class BookService {

    @Autowired
    BookRepository bookRepository;

    public void test() {
        bookRepository.save(new Book(5, "my book", "ykleee", new java.util.Date()));
        List<Book> bookList = (List<Book>) bookRepository.findAll();
        bookRepository.deleteById(5);
    }
}
```


» Entity Sample Code

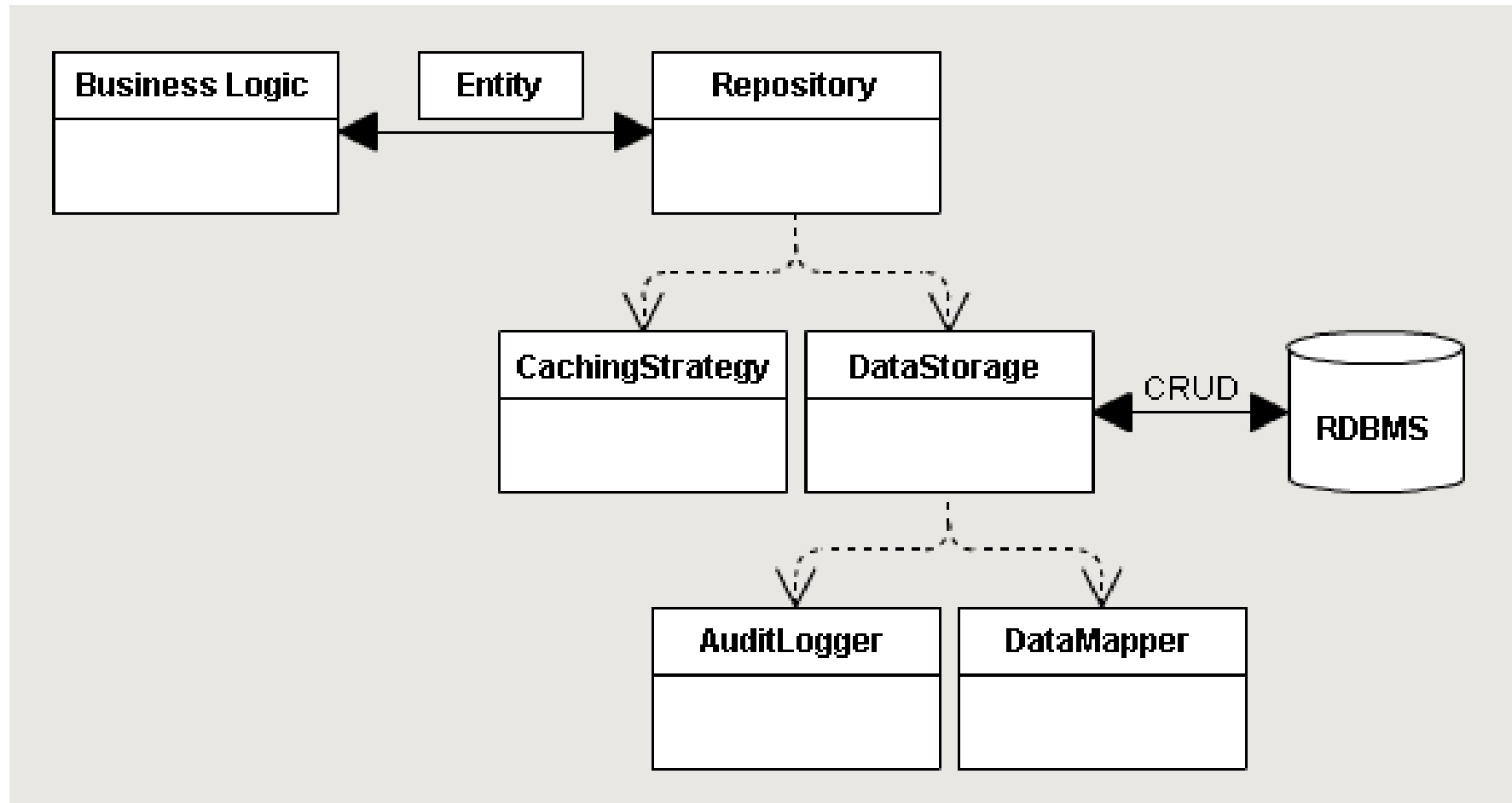
```
@Entity
@Getter
@NoArgsConstructor(access= AccessLevel.PROTECTED)
@AllArgsConstructor
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", updatable= false)
    private long id;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "author")
    private String author;

    @CreationTimestamp
    @Column(name = "published", columnDefinition =
        "TIMESTAMP DEFAULT CURRENT_TIMESTAMP", updatable = false)
    @Temporal(TemporalType.TIMESTAMP)
    private Date published;
}
```

» Entity sample



» Example

```
@Document(collection = "person")
public class Person {

    @Id
    private String id;
    private String firstName;
    private String lastName;
    private int age;
    private String gender;

    public interface PersonRepository extends
    MongoRepository<Person, String> {

        public List<Person> findByLastName(String lastName);
        public List<Person> findByAgeGreaterThan(int age);

    }
}
```

» Example of application.properties

```
spring.application.name=second
spring.datasource.url=jdbc:mysql://mysql:3306/web?useSSL=false&server
Timezone=UTC&characterEncoding=UTF-8
spring.datasource.username=root
spring.datasource.password=rootpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
# Thymeleaf Disable cache (used at
development)spring.thymeleaf.cache=false
server.port=8090
```

Why is it right for cloud native development

- ✓ Lightweight + Quick Run (Lightweight Editor)
- ✓ Memory usage and loading are much lower than traditional IDE (e.g., IntelliJ IDEA, Eclipse IDE)
- ✓ Big advantage as fast restart/access becomes more important, especially in container and cloud development environments
- ✓ In Kubernetes, MSA environments, VS Code is much more efficient than heavy IDE

Why is it right for cloud native development

- ✓ Extension-based architecture
- ✓ Lightweight and built to add only the features you need
- ✓ Representative Extensions:
 - ✓ Python extension for VS Code
 - ✓ Spring Boot Extension Pack
 - ✓ Docker extension for VS Code
 - ✓ Kubernetes extension for VS Code
- ✓ This means "IDE + DevOps + Cloud Tool" can be integrated into one

Why is it right for cloud native development

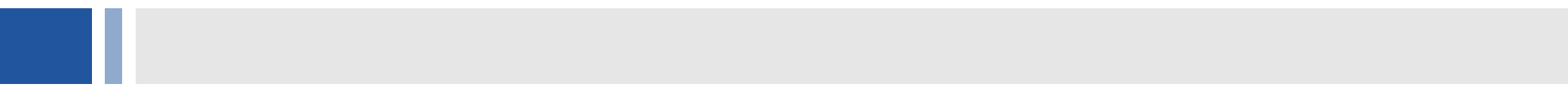
- ✓ Cloud / Container Friendly
- ✓ Remote Development Support:
- ✓ SSH connection
- ✓ Internal development of containers
- ✓ WSL Interworking
- ✓ DevContainer, Codespaces Support → Environmental Consistency

Why is it right for cloud native development

- ✓ Git Integration (DevOps Friendly)
- ✓ Built-in Git UI by default
- ✓ Git Operation: commit / push / diff / branch management
- ✓ Connect with GitHub, GitLab naturally

Extension packs to install (Ctrl+Shift+X)

- ✓ Extension pack for Java
- ✓ Docker
- ✓ Spring boot extension pack
- ✓ Container tools



감사합니다

THANK YOU